

Introducción a Regex (Expresiones Regulares) en Python

¿Qué es Regex?

Las expresiones regulares (o regex) son una herramienta muy poderosa que sirve para buscar, extraer o reemplazar texto de forma flexible. Son muy útiles en el Procesamiento del Lenguaje Natural (PLN) cuando queremos:

- Buscar palabras o frases específicas en un texto.
- Limpiar datos (quitar puntuación, espacios extra, etc.)
- Identificar patrones (como fechas, correos electrónicos, etc.)

No se busca una palabra exacta, sino un patrón de forma general. Son como una minilengua con reglas para identificar cosas en el texto.

Las expresiones regulares te ayudarán a buscar patrones lingüísticos en grandes corpus de texto. Además, son especialmente útiles para limpiar textos, buscar estructuras o crear filtros de información.

Aunque al principio pueden parecer extrañas, ¡con la práctica se vuelven una herramienta indispensable!

¿Por qué son útiles para la programación y el PLN?

En Procesamiento del Lenguaje Natural (PLN), trabajamos con mucho texto. Regex sirve para:

- Limpiar textos antes de analizarlos.
- Detectar palabras que tienen una forma concreta (verbos, nombres propios, números...).
- Extraer información útil de textos grandes: correos, fechas, cifras, palabras clave...

¿Y por qué son importantes para un futuro lingüista computacional?

Un lingüista computacional o un lingüista que trabaja con traducción automática, corpus lingüísticos, corrección ortográfica, chatbots, búsqueda de información... necesita extraer patrones del lenguaje de forma rápida y precisa. Y eso lo hace con ayuda de herramientas como Regex.

Por ejemplo:

- Crear una herramienta que busque verbos en infinitivo dentro de un corpus.
- Analizar estilos lingüísticos en traducciones.
- Encontrar errores frecuentes en textos escritos.

Aunque ahora lo estamos viendo con ejemplos simples, estos conceptos se usan en muchas herramientas profesionales, como motores de traducción automática, sistemas de corrección, minería de textos, etc.

```
import re

texto = "Mi gato duerme todo el día."

resultado = re.search("gato", texto)

print(resultado)
```

🧠 ¿Qué ha pasado aquí?

`re.search()` busca la primera aparición del patrón (en este caso "gato") dentro del texto. Si lo encuentra, devuelve un objeto especial (`Match`); si no, devuelve `None`.



Empecemos a trabajar con Regex

```
if resultado:
    print("¡Encontré la palabra 'gato'!")
else:
    print("No encontré la palabra 'gato'.")
```

Probemos ahora a buscar 'perro'.

```
texto = "Mi gato duerme todo el día."
resultado = re.search("perro", texto)
print(resultado)

if resultado:
    print("¡Encontré la palabra 'gato'!")
else:
    print("No encontré la palabra 'gato'.")
```



Búsqueda de palabras que empiezan por una letra concreta

```
texto2 = "El coche y la casa están cerca del campo."

palabras_c = re.findall(r"\bc\w+", texto2)

print("Palabras que empiezan por 'c':", palabras_c)
```



Reemplazar palabras con Regex

```
texto3 = "María tiene un perro. El perro se llama Max."

nuevo_texto = re.sub("perro", "gato", texto3)

print(nuevo_texto)
```

Limpiar puntuación

```
texto4 = "¡Hola! ¿Cómo estás? Estoy bien, gracias."
```

```
texto_limpio = re.sub(r"[\w\s]", "", texto4)
```

```
print("Texto sin puntuación:", texto_limpio)
```

Buscar todas las apariciones de una palabra

```
texto5 = "El gato está en el tejado. El gato duerme."
```

```
apariciones = re.findall("gato", texto5)
```

```
print("Número de veces que aparece 'gato':", len(apariciones))
```

```
print("Lista de coincidencias:", apariciones)
```

Buscar correos electrónicos

```
texto6 = "Puedes escribir a juan@example.com o a maria@gmail.com."
```

```
correos = re.findall(r"\w+@\w+\.", texto6)
```

```
print("Correos encontrados:", correos)
```

Encontrar todas las palabras que terminan en "-ar" (como en los verbos en infinitivo)

```
texto = "Cantar, bailar, comer y vivir son verbos comunes."
```

```
palabras_ar = re.findall(r"\b\w+ar\b", texto)
```

```
print("Palabras que terminan en 'ar':", palabras_ar)
```

Contar cuántas palabras hay en una frase

```
frase = "La lingüística computacional es fascinante."
```

```
palabras = re.findall(r"\b\w+\b", frase)
```

```
print("Palabras encontradas:", palabras)
```

```
print("Número total de palabras:", len(palabras))
```

Detectar mayúsculas iniciales (nombres propios, principios de oración)

```
texto = "Madrid es la capital de España. Juan estudia Lenguas Modernas."
```

```
mayusculas = re.findall(r"\b[A-Z]\w+", texto)
```

```
print("Palabras con inicial mayúscula:", mayusculas)
```

✓ Verificar si una palabra aparece al principio de una frase

```
frase = "Hola, ¿cómo estás?"
```

```
if re.match(r"^Hola", frase):  
    print("La frase empieza por 'Hola'")  
else:  
    print("La frase NO empieza por 'Hola'")
```

✓ Extraer números en un texto

```
texto = "Tengo 2 gatos, 1 perro y 3 peces."
```

```
numeros = re.findall(r"\d+", texto)
```

```
print("Números encontrados:", numeros)
```

```
texto = "Tengo 2 gatos, 1 perro y 3 peces. Mi teléfono es el  
679483750."
```

```
numeros = re.findall(r"\d{4,}", texto)
```

```
print("Números encontrados:", numeros)
```

```
texto = "Tengo 2 gatos, 1 perro y 3 peces. Mi teléfono es el 679483750  
y mi pin 1234."
```

```
numeros_prueba1 = re.findall(r"\d{4}", texto)
```

```
print("Números encontrados en la prueba 1:", numeros_prueba1)
```

```
numeros_prueba2 = re.findall(r"\b\d{4}\b", texto)
```

```
print("Números encontrados en la prueba 2:", numeros_prueba2)
```

```
texto = "Tengo 2 gatos, 1 perro y 3 peces. Mi teléfono es el 679483750  
y mi pin 1234."
```

```
texto_sin_numeros = re.sub(r"\d+", "", texto)
```

```
print("Texto sin números:", texto_sin_numeros)
```

📌 Resumen para recordar

Expresión	Significado
.	Cualquier carácter (excepto salto de línea)

Expresión	Significado
<code>\w</code>	Letras, números o guion bajo
<code>\s</code>	Espacio en blanco
<code>\d</code>	Dígito (0-9)
<code>+</code>	Uno o más
<code>*</code>	Cero o más
<code>\b</code>	Límite de palabra
<code>[]</code>	Conjunto de caracteres
<code>^</code>	Comienzo de línea o negación en corchetes
<code>\$</code>	Final de línea



Actividades propuestas

Recuerda que, para ambas actividades, es necesario importar el módulo `re`.



Actividad 1: Normalización de un corpus y recuento léxico

Objetivo: Practicar la limpieza de datos (`re.sub()`) y el conteo de palabras (`re.findall()`), tareas fundamentales en la preparación de textos para el análisis lingüístico.

Situación: Ha recibido un fragmento de una entrevista histórica que contiene signos de exclamación, interrogación, y otros símbolos que no deben ser contados como palabras en un análisis de frecuencia. Necesita limpiar el texto y luego contar cuántas palabras reales quedan.

Conocimientos que se aplicarán:

- Uso de `re.sub()` para reemplazar patrones.
- Uso del patrón de negación `[^\w\s]` para eliminar la puntuación.
- Uso de `re.findall()` para extraer todas las palabras.
- Uso del patrón `\b\w+\b` para definir una palabra.

Instrucciones:

1. Declara la siguiente cadena de texto como la variable `entrevista_sucia`: "¡Increíble! ¿Podemos decir que 'Internet' es el futuro? El 20% de los usuarios dijo: ¡SÍ! La tecnología avanza a pasos agigantados."
2. Utiliza `re.sub()` y el patrón `[^\w\s]` para eliminar toda la puntuación, dejando solo letras, números, y espacios. Guarda el resultado en `entrevista_limpia`.
3. Imprime el contenido de `entrevista_limpia`.
4. Utiliza `re.findall()` con el patrón `r"\b\w+\b"` para encontrar todas las palabras y números que quedan en `entrevista_limpia`. Guarda la lista en la variable `tokens_finales`.

5. Imprime el número total de tokens (palabras/cifras) encontrados utilizando la función `len()`.

Actividad 2: Extracción de estructuras nominales (detección de mayúsculas)

Objetivo: Usar expresiones regulares para identificar palabras que comienzan con mayúscula, lo cual es útil para la detección preliminar de nombres propios, lugares o el inicio de oraciones.

Situación: Estás realizando un estudio preliminar sobre la aparición de entidades nombradas (personas, lugares, organizaciones) en un texto. Una heurística inicial y sencilla es buscar todas las palabras que comienzan con mayúscula.

Conocimientos que se aplicarán:

- Uso de `re.findall()` para extraer todas las coincidencias.
- Uso del límite de palabra `\b`.
- Uso del conjunto de caracteres `[A-Z]` para especificar una letra mayúscula.
- Uso de `\w+` para asegurar que el resto de la palabra sea capturada.

Instrucciones:

1. Declara la siguiente cadena de texto como la variable `fragmento_noticia`: "La Real Academia Española publicó la noticia. Madrid y Barcelona lideran el cambio. En 2023, la RAE fue premiada."
2. Utiliza `re.findall()` con el patrón `r"\b[A-Z]\w+"` para encontrar todas las palabras que comienzan con una letra mayúscula (A-Z). Almacena la lista resultante en `entidades_potenciales`.
3. Imprime la lista de `entidades_potenciales`.
4. Utiliza un segundo patrón, más estricto, que encuentre solo aquellos elementos que son acrónimos o números de dos o más dígitos (`\d+` o `[A-Z]+`). Tarea adicional: Encuentra solo los números de dos o más dígitos (usando `\d+`).
5. Imprime la lista de números encontrados.



Pregunta de reflexión

¿Identificas algún problema en el resultado de la segunda actividad? Desarrolla tu respuesta.